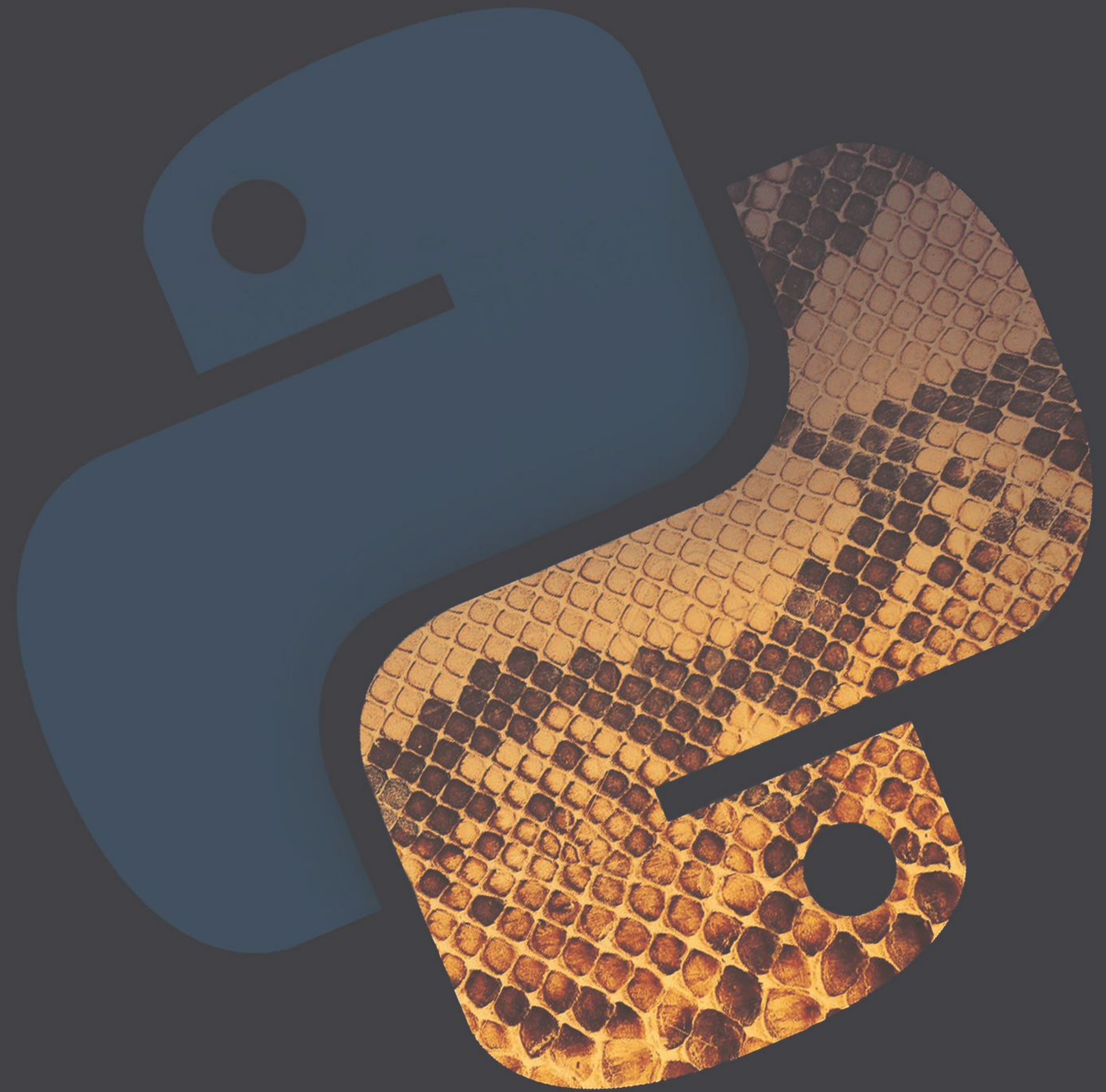


Introducción a Python

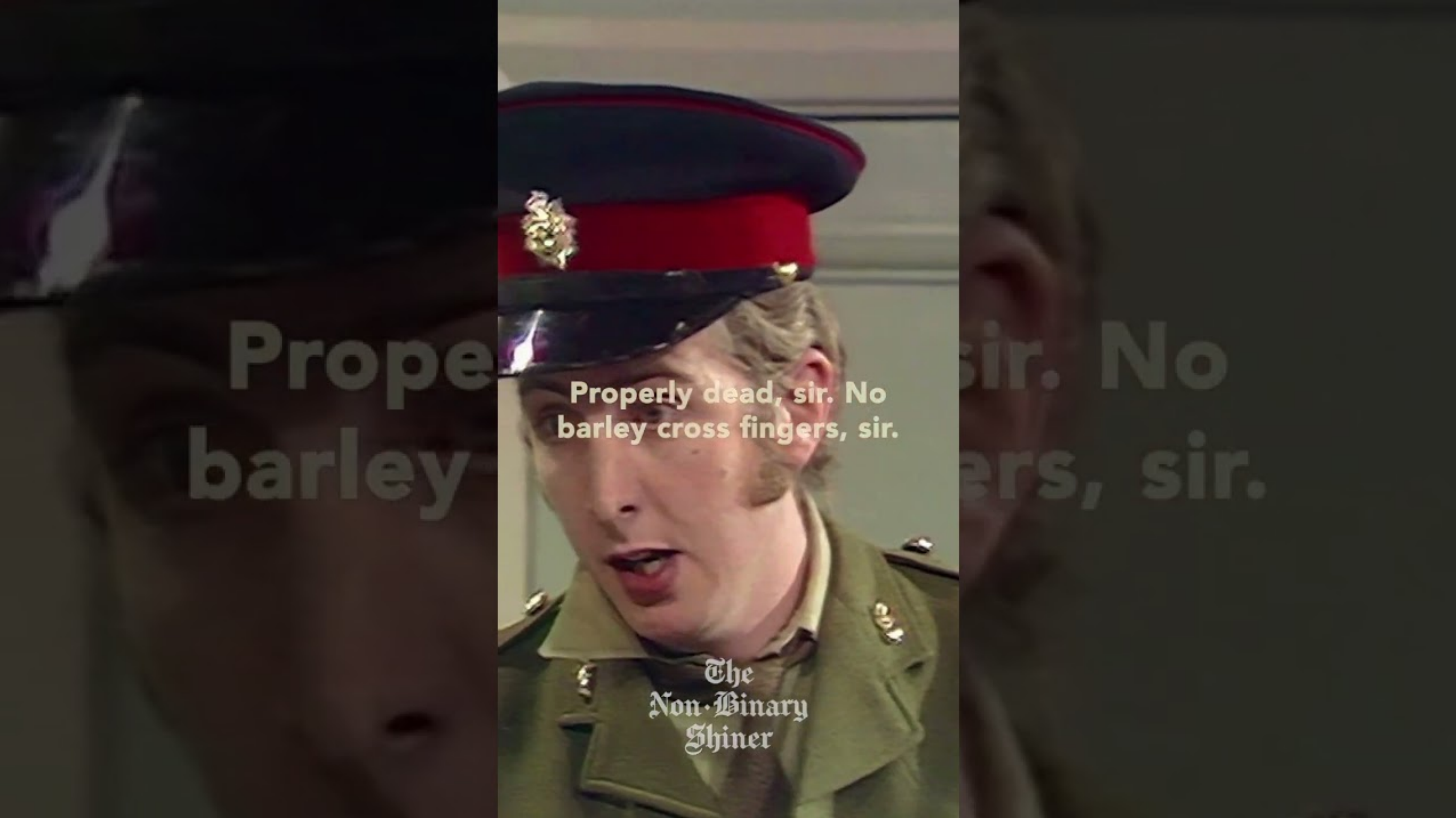


Física Computacional 2024-2025
Serena Di Santo

Introducción a Python

Python's developers aim for it to be fun to use. This is reflected in its name—a tribute to the British comedy group [Monty Python](#)^[83]—and in occasionally playful approaches to tutorials and reference materials, such as the use of the terms "spam" and "eggs" (a reference to [a Monty Python sketch](#)) in examples, instead of the often-used "foo" and "bar".^{[84][85]} A common neologism in the Python community is *pythonic*, which has a wide range of meanings related to program style. "Pythonic" code may use Python [idioms](#) well, be natural or show fluency in the language, or conform with Python's minimalist philosophy and emphasis on readability. Code that is difficult to understand or reads like a rough transcription from another programming language is called *unpythonic*.^{[86][87]}





Prope
barley

Properly dead, sir. No
barley cross fingers, sir.

The
Non-Binary
Shiner

sir. No
ers, sir.

Introducción a Python

Python's developers aim for it to be fun to use. This is reflected in its name—a tribute to the British comedy group [Monty Python](#)^[83]—and in occasionally playful approaches to tutorials and reference materials, such as the use of the terms "spam" and "eggs" (a reference to [a Monty Python sketch](#)) in examples, instead of the often-used "foo" and "bar".^{[84][85]} A common neologism in the Python community is *pythonic*, which has a wide range of meanings related to program style. "Pythonic" code may use Python [idioms](#) well, be natural or show fluency in the language, or conform with Python's minimalist philosophy and emphasis on readability. Code that is difficult to understand or reads like a rough transcription from another programming language is called *unpythonic*.^{[86][87]}



SimuVerse: Physics Unraveled



SimuVerse: Physics Unraveled



The Pythonic Code Award

For adhering to best practices in
formatting, naming, and style conventions

<https://inventwithpython.com/beyond/chapter3.html>

Introducción a Python



- * *Python es un lenguaje de programación **interpretado** (no necesita compilación)*
- * *Debido a su facilidad de uso y al gran numero de bibliotecas disponibles, es **muy usado** en la actualidad en una multitud de campos como ciencia de datos, machine learning, desarrollo web, visualización de datos....*
- * *Al ser un lenguaje interpretado es **menos eficiente** (más lento) que lenguajes compilados como C, C++ o Fortran. A cambio, es mucho mas comodo de usar*

Instalamos Python (Anaconda)

Anaconda es una plataforma de código abierto que se especializa en la gestión de entornos y paquetes Python.

Proporciona un administrador de paquetes llamado **conda**, que simplifica la instalación y actualización de bibliotecas Python y sus dependencias.

Instalamos Python (Anaconda)

```
> conda --version  
conda 22.11.1
```

if “command not found”

<https://docs.anaconda.com/getting-started/>

```
> conda --help
```

```
usage: conda [-h] [-V] command ...
```

conda is a tool for managing and deploying applications, environments and packages.

Options:

```
[. . .]
```

```
create
```

Create a new conda environment from a list of specified packages.

```
install
```

Installs a list of packages into a specified conda environment.

Creamos un Environment

Anaconda permite crear entornos Python aislados para cada proyecto, lo que garantiza que las dependencias no entren en conflicto entre sí. Esto es fundamental para evitar problemas de compatibilidad entre versiones de paquetes

```
> conda create -n snakes
```

```
> conda info --envs
```

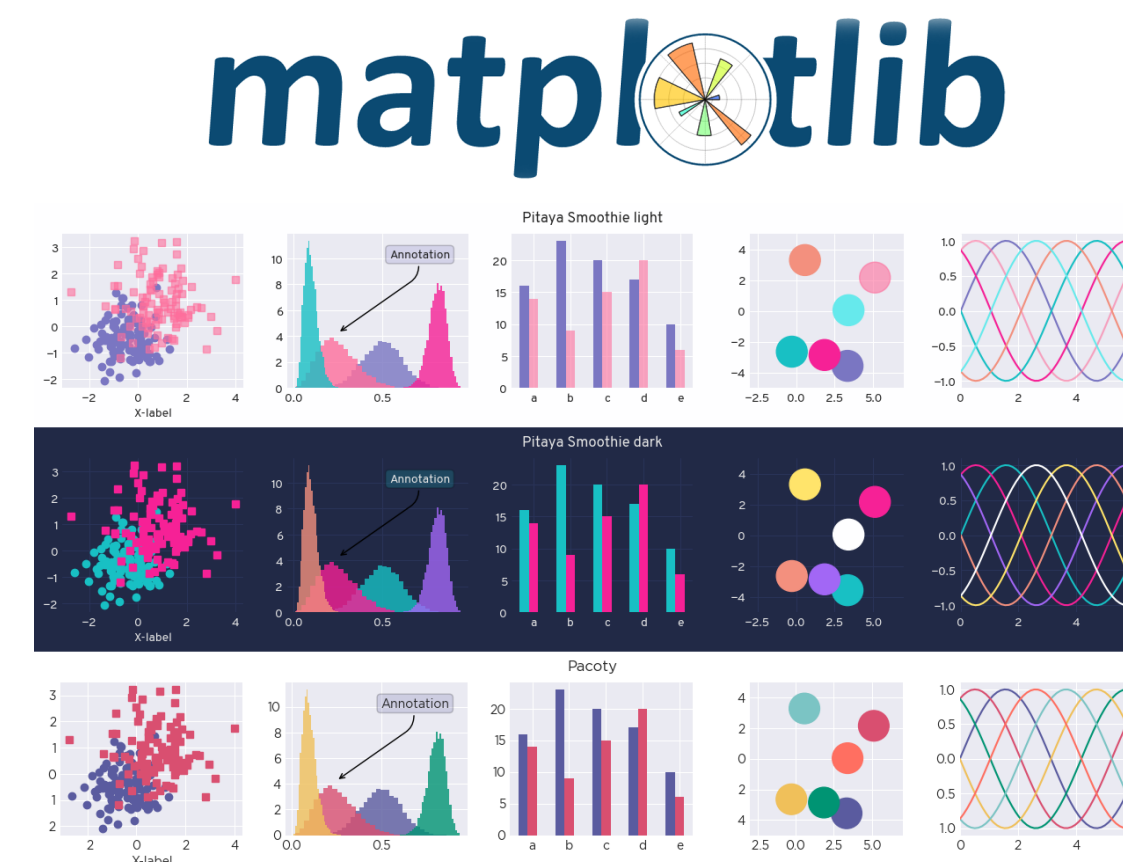
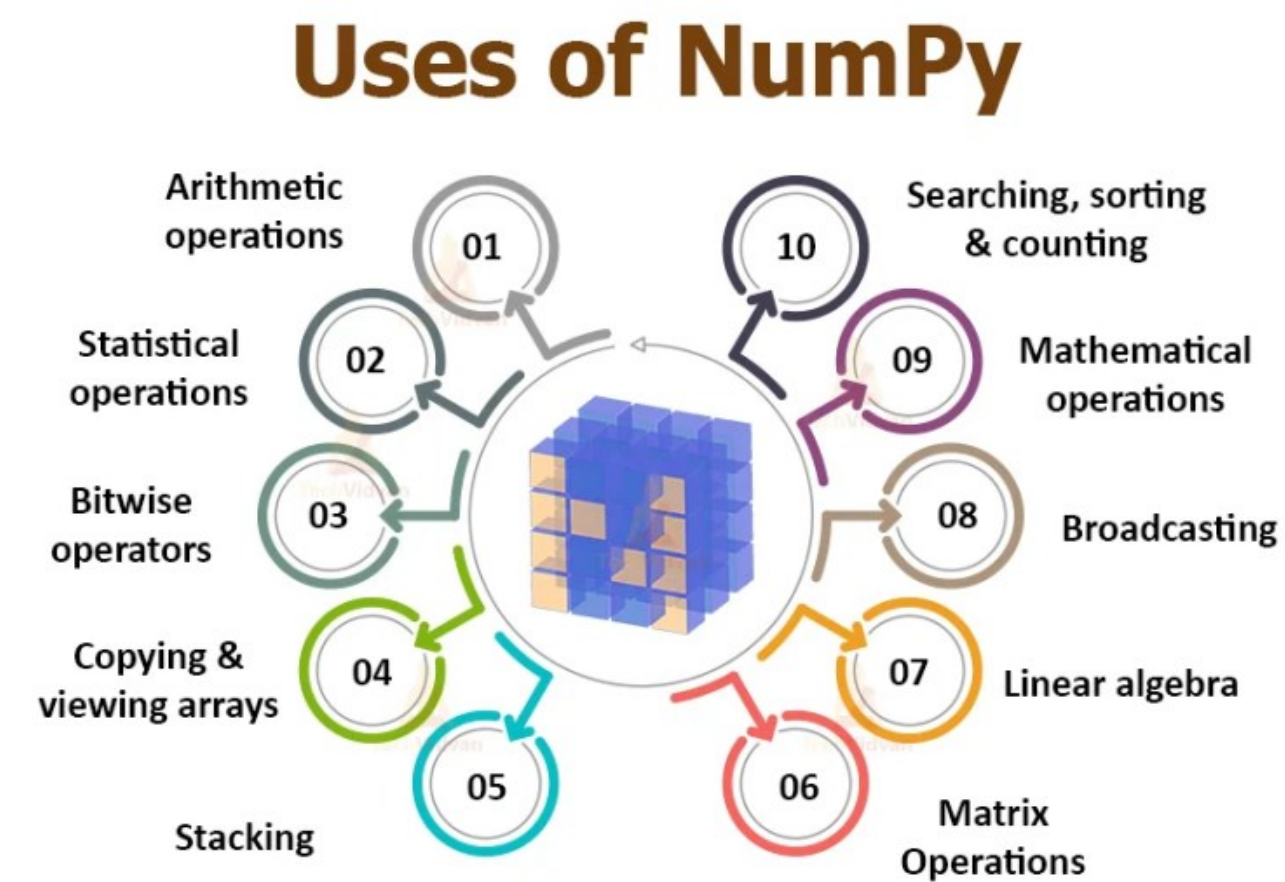
```
base          *  /Users/serenadisanto/opt/anaconda3  
snakes        /Users/serenadisanto/opt/anaconda3/envs/snakes
```

```
> conda activate snakes
```

<https://docs.conda.io/projects/conda/en/stable/user-guide/cheatsheet.html>

Instalamos librerías útiles

```
> conda install numpy matplotlib jupyter
```



Extensiones de Python en VSC

The image displays three overlapping screenshots of the Visual Studio Code (VSC) Extensions Marketplace, illustrating the process of installing Python-related extensions.

Top Screenshot: Python Extension

- Search:** The search bar contains "python".
- Results:** The first result is "Python" by Microsoft, with a download time of 137ms. Below it is "Python Debugger" by Microsoft (29ms) and "Python" by shiro (120K stars, 5 stars).
- Details:** The "Python" extension page is shown, version v2024.2.1, by Microsoft (microsoft.com), with 115,093,46 downloads. It includes buttons for "Disable", "Uninstall", and "Switch to Pre-Release". A note states: "This extension is enabled globally."

Middle Screenshot: Python Debugger Extension

- Search:** The search bar contains "python".
- Results:** The first result is "Python" by Microsoft (137ms). The second result is "Python Debugger" by Microsoft (29ms), which is highlighted.
- Details:** The "Python Debugger" extension page is shown, version v2024.2.0, by Microsoft (microsoft.com), with 17,506,720 downloads. It includes buttons for "Disable", "Uninstall", and "Switch to Pre-Release". A note states: "This extension is enabled globally."

Bottom Screenshot: Jupyter Extension

- Search:** The search bar contains "jupyter".
- Results:** The first result is "Jupyter" by Microsoft, with a download time of 762ms. Below it are "Jupyter Keymap" by Microsoft and "Jupyter Slide Sh..." by Microsoft (2ms).
- Details:** The "Jupyter" extension page is shown, version v2024.1.1, by Microsoft (microsoft.com), with 75,764,639 downloads. It includes buttons for "Disable", "Uninstall", and "Switch to Pre-Release". A note states: "This extension is enabled globally."

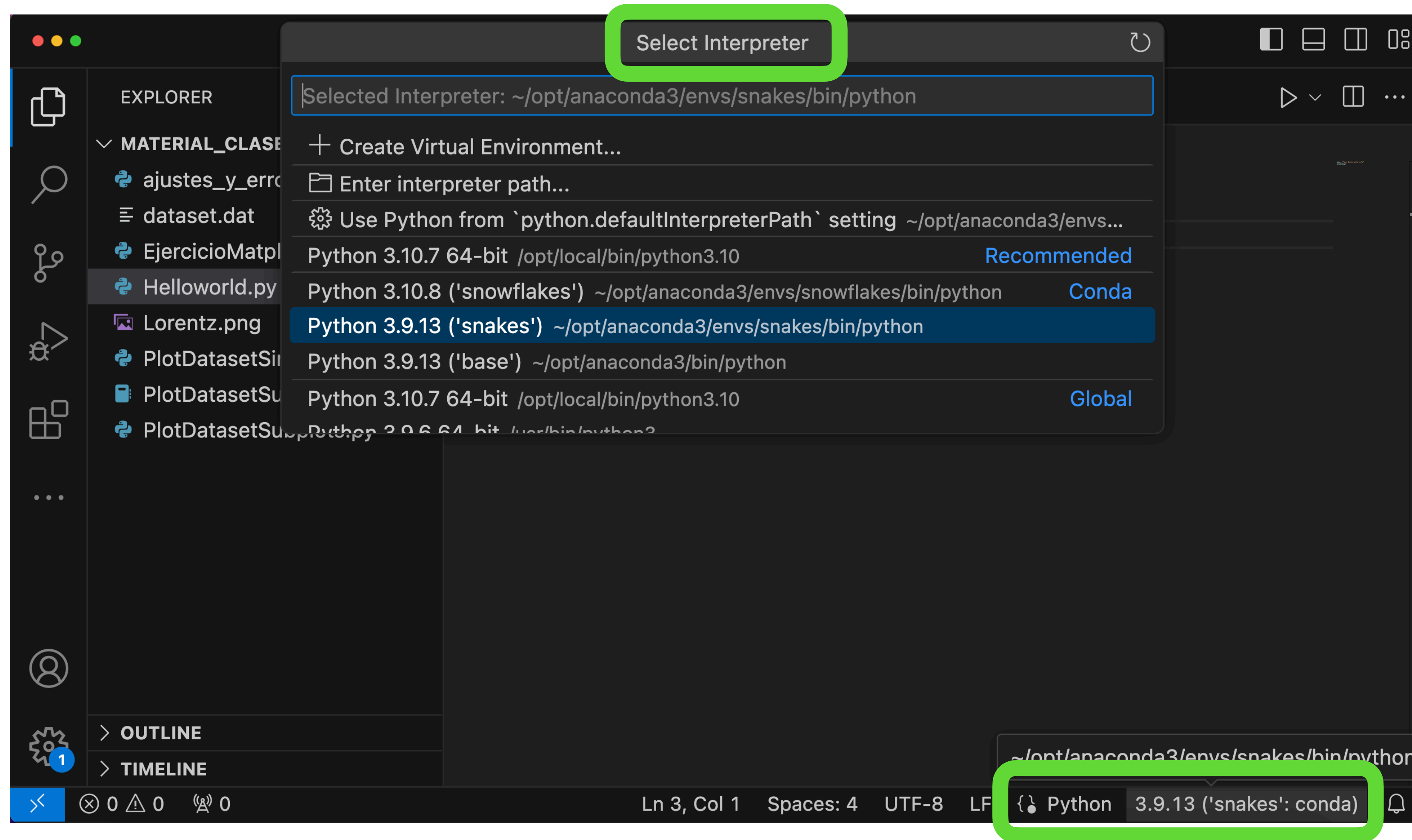
Primeros pasos con Python

- * *No hay que declarar las variables*
- * *No hay que asignar tipos a las variables (se asignan de forma dinámica e implícita)*
- * *Las líneas no terminan en ;*
- * *Los comentarios empiezan por #*
- * *Las cadenas de caracteres se definen de forma nativa*

```
 helloworld.py > ...  
1  msg = "\n** Hello world **\n"  
2  print(msg)  
3  
```

Seleccionamos el Environment

>Python: Select Interpreter



La función print()

- * *Para manipular texto podemos usar tanto comillas simples ('...') como dobles*
- * *Se pueden introducir varios argumentos en la función print(), separados por comas*

```
mensaje1='Hola'
mensaje2="mundo de Python!"
variable=5
print(mensaje1, mensaje2)
print("el valor de la variable es ", variable)
```

✓ 0.0s

```
Hola mundo de Python!
el valor de la variable es  5
```

La función print()

print() Signature

Python Syntax

```
print(*objects, sep=" ", end="\n", file=None, flush=False)
```

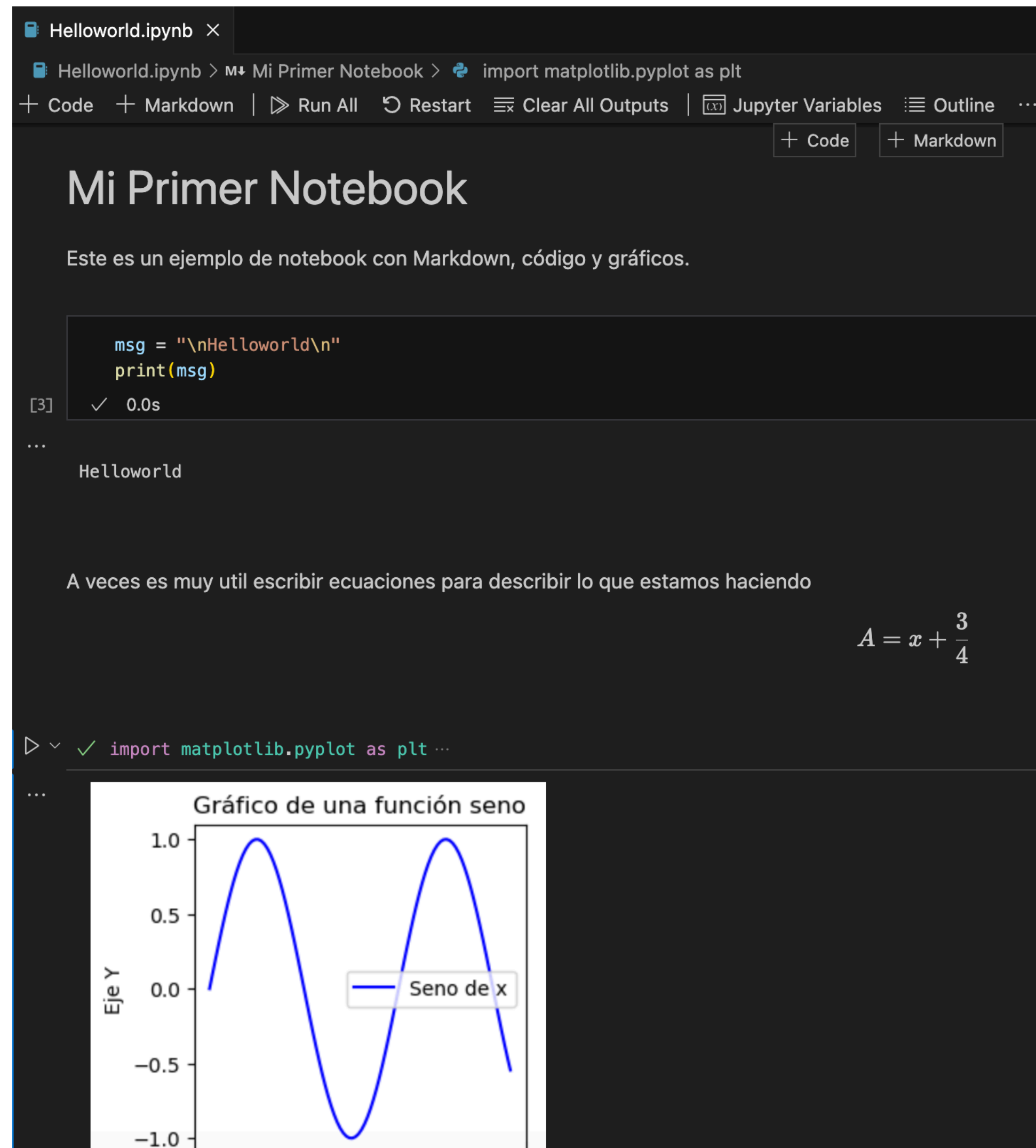
```
print("Python", "is", "fun", sep=" - ", end="!!\n\n")  
print("Python", "is", "fun", sep="..")
```

✓ 0.0s

Python - is - fun!!

Python..is..fun

El notebook de Python



- * *Los notebooks de Python (también conocidos como **Jupyter notebooks**) son entornos de computación interactivos donde puedes escribir y ejecutar código Python de manera secuencial. Permiten combinar código, texto, visualizaciones y ecuaciones en un solo documento.*

El notebook de Python

The screenshot shows a Jupyter Notebook interface with a dark theme. At the top, there's a tab labeled 'Helloworld.ipynb'. Below it, the notebook title 'Mi Primer Notebook' is displayed. A toolbar contains buttons for '+ Code', '+ Markdown', 'Run All', 'Restart', 'Clear All Outputs', 'Jupyter Variables', and 'Outline'. The notebook content includes a Markdown cell with the text 'Este es un ejemplo de notebook con Markdown, código y gráficos.', followed by a Code cell containing Python code:

```
msg = "\nHelloworld\n"
print(msg)
```

. The output of the code cell is 'Helloworld'. Below that is another Markdown cell with the text 'A veces es muy util escribir ecuaciones para describir lo que estamos haciendo' and a LaTeX equation $A = x + \frac{3}{4}$. At the bottom, there's a Code cell with

```
import matplotlib.pyplot as plt
```

 and a corresponding plot titled 'Gráfico de una función seno' showing a sine wave. The plot has a legend entry 'Seno de x'. The 'Run' button (a play icon) in the toolbar is circled in blue.

Inserta una celda de tipo **Markdown**, para escribir texto formateado

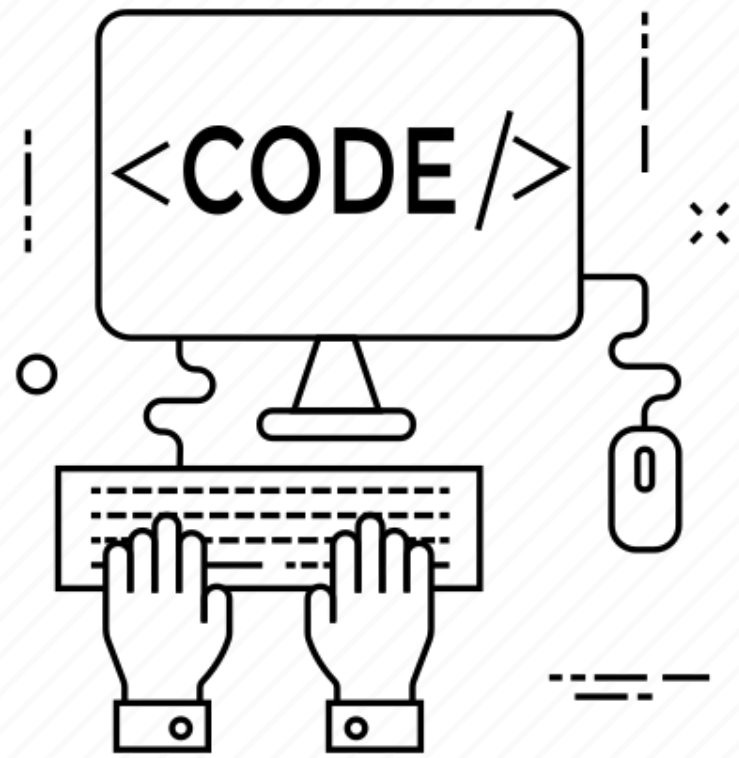
Inserta una celda de tipo **Code**, para escribir código en Python

En las celdas Markdown podemos escribir ecuaciones en LaTeX

Las celdas pueden ejecutarse de manera independiente

Las celdas pueden “colapsarse”

Manos a la obra



- * *Escribir y ejecutar el programa **Holamundo.py***
- * *Escribir el programa **Holamundo.ipynb** que contenga al menos una celda **Markdown** y una celda **Code***

Comentarios

- * *Los comentarios empiezan por #*
- * *Los comentarios multilinea están comprendidos entre triples comillas ''' o """*

```
# This is a single-line comment.  
# Comments help explain code but do not affect execution.  
  
# Below, we define a function to add two numbers.  
def add_numbers(a, b):  
    """  
    This is a multi-line comment (also called a docstring).  
    It explains what the function does.  
  
    This function takes two numbers as input and returns their sum.  
    """  
    return a + b # Returns the sum of a and b
```

```
help(add_numbers)
```

✓ 0.0s

Help on function add_numbers in module __main__:

```
add_numbers(a, b)  
    This is a multi-line comment (also called a docstring).  
    It explains what the function does.  
  
    This function takes two numbers as input and returns their sum.
```


Operaciones aritméticas

- * *Las operaciones pueden involucrar números enteros y reales.*
- * *La división siempre devuelve un numero float.*
- * *Ejemplos:*

$$5/2*3.0=7.5$$

$$3.0*5/2=7.5$$

$$5./2.*3.0=3.0*5./2.=7.5$$

```
print(8 / 2) # 4.0 (no 4, sino 4.0)
```

Operaciones aritméticas

- * Además de las operaciones básicas `+`, `-`, `*`, `/`
- * el símbolo `**` indica una potencias, la sintaxis es: `base ** exponente`
- * El resto de la división entera se obtiene con `%`
- * Para la división entre enteros se usa el operador `//`
- * Python también soporta números y variables complejas. La unidad imaginaria se escribe como `1j`

```
print(5+10*2)
print(4**3)
print(18//4)
print(18%4)
print(1j*1j)
```

✓ 0.0s

25

64

4

2

(-1+0j)

Operaciones aritméticas: numpy

- * **NumPy (Numerical Python)** es una biblioteca fundamental para la computación numérica en Python. Proporciona soporte para funciones matemáticas avanzadas, arrays multidimensionales y herramientas para trabajar con datos numéricos de manera eficiente.
- * Para importar la biblioteca numpy se usa:

import numpy as np

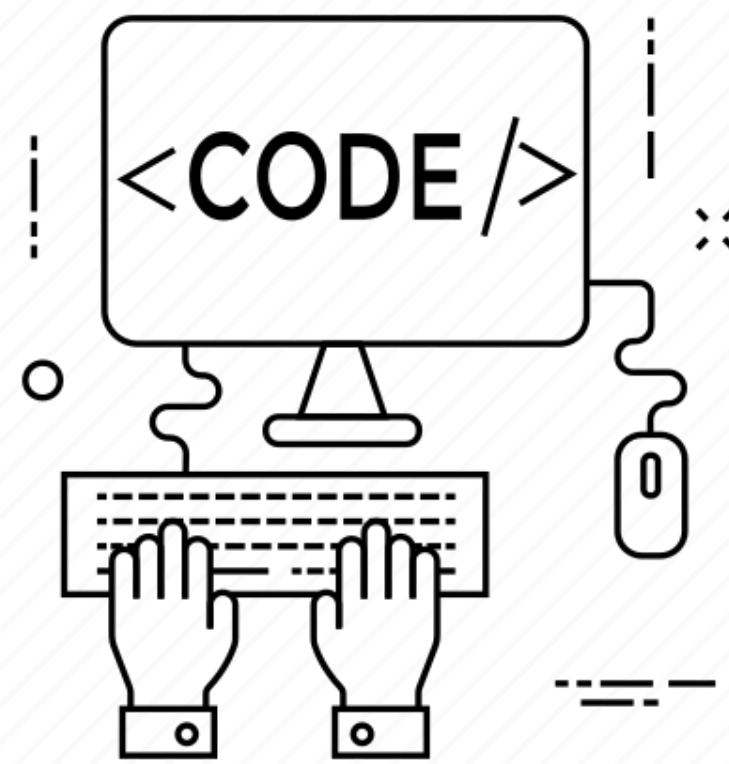
El alias **np** es un estándar en la comunidad para escribir código más corto.

Las funciones de numpy se importan con *alias.funcion(..)*

```
import numpy as np

x=90
np.sin(x)+np.sqrt(x)
```


Manos a la obra



- * *Abrir un notebook en Python llamado “ejercicios_Python.ipynb”. Escribir una celda Markdown que contenga el primer problema:*
- * *Un cohete es lanzado desde una altura inicial H_0 con velocidad inicial V_0 y una aceleración vertical A . Escribir en LaTeX las ecuaciones que describen la velocidad y altura del cohete al tiempo T .*
- * *Escribir una celda Code que asigne el valor $-9.807 \text{ (m/s}^2\text{)}$ a A , 150 (m) a H_0 , 100.00 (m/s) a V_0 y 5.0 (s) a T , calcule los valores de H y v al instante T y los imprima por pantalla.*

Control de flujo

Control de flujo: if, else

- * La proposición if-else se utiliza para expresar decisiones. La síntesis formal es:

```
if condición:  
    orden_1  
    orden_2  
else:  
    orden_3
```

- * CUIDADO: la **indentación** es esencial para indicar qué código pertenece a una **función**, **bucle**, **condición**, etc. En lugar de usar llaves `}` como en C++, Python utiliza la indentación para estructurar el código.
- *Cuál es la diferencia entre estos dos códigos?

```
if condición:  
    orden_1  
orden_2
```

```
if condición:  
    orden_1  
orden_2
```


Control de flujo: if, else

- * Se puede usar la orden **elif** para ejecutar ordenes si no se ha cumplido la condición anterior y se cumple una nueva.

```
x=2
if x>5:
    print('mayor de 5\n')
elif x<3:
    print('menor de 3\n')
else:
    print('entre 3 y 5\n')
```

Control de flujo: if, else

- * La proposición a evaluar se puede definir mediante el uso de operadores ($\geq$, $\leq$, $=$, $\neq$, $\&$, $|$).
- * Para usar proposiciones compuestas hay que usar paréntesis.

```
# Definimos dos variables para las comparaciones
a = 10
b = 20
c = 10

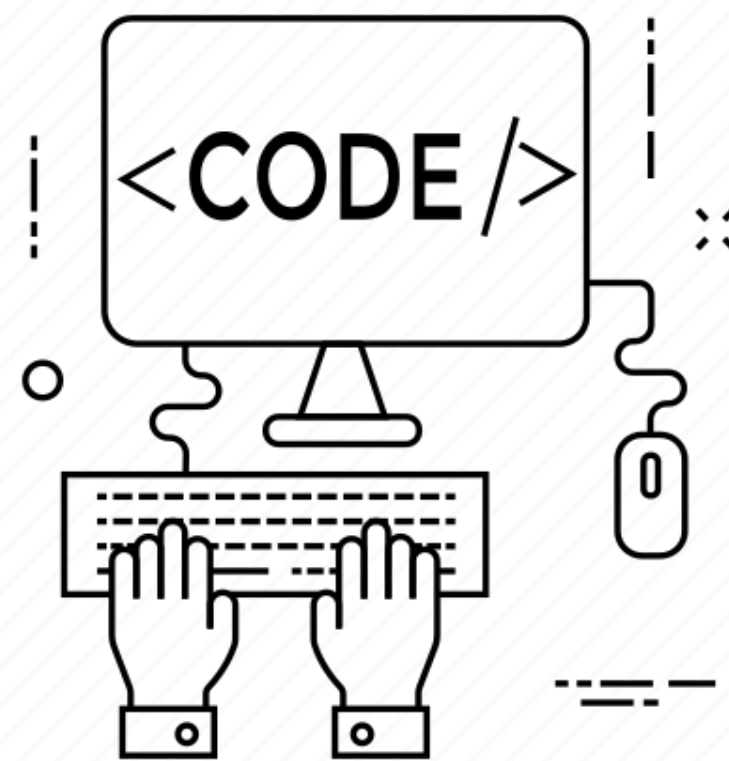
if a == c:
    print("a es igual a c") # Se ejecuta porque 10 == 10

if a != b:
    print("a no es igual a b") # Se ejecuta porque 10 != 20

if (a == 10) & ((a + c) == b):
    print("a es igual a 10 y la suma de a y c es igual a b") # Se ejecuta porque a = 10 y 10 + 10 = 20

if (a == 4) | ((a + c) == b):
    print("a es igual a 4 o la suma de a y c es igual a b o ambas") # Se ejecuta porque 10 + 10 = 20
```

Manos a la obra



- * *Añadir una celda Markdown al notebook ejercicios_Python.ipynb que introduzca el siguiente ejercicio para practicar el uso de if.*
- * *Asignar un valor 9 a la variable x, determinar si esa variable es par, si lo es imprimir en pantalla:
“la variable es par”;
si es impar, que determine si es multiplo de 3 y si lo es imprima en pantalla:
“la variable es impar y multiplo de 3”*

Control de flujo: for

* La sintaxis general es: *for variable in secuencia:*

orden_1

orden_2

* La función *range(n)* se utiliza para generar una secuencia de números (de 0 a $n-1$), y es muy común en bucles for, cuando necesitamos hacer algo un número específico de veces.

* La función *range()* se puede usar especificando además del límite superior, también el límite inferior y el paso: *range(start, stop, step)*

```
for i in range(5):  
    print(i)
```

✓ 0.0s

0
1
2
3
4

```
for i in range(5, 26, 5): # Desde 5 (incluido) hasta 26, con paso de 5  
    print(i)
```

✓ 0.0s

5
10
15
20
25

Control de flujo: for

* La sintaxis general es: *for variable in secuencia:*

orden_1

orden_2

* También se puede usar *for* para iterar sobre una lista, por ejemplo:

```
Lista_alumnas = ["Alba", "Jimena", "Marta", "Paula"]
```

```
for alumna in Lista_alumnas:  
    print(alumna)
```

✓ 0.0s

Alba

Jimena

Marta

Paula

Control de flujo: for

- * *A veces es útil acceder tanto al elemento de la lista como a su índice dentro del bucle for. En ese caso podemos usar la función `enumerate()`:*

for indice, variable in enumerate (secuencia):

orden_1

orden_2

```
Lista_alumnas = ["Alba", "Jimena", "Marta", "Paula"]
```

```
for ind, alumna in enumerate(Lista_alumnas):  
    print(alumna, "aparece en posición", ind, "en la lista")
```

✓ 0.0s

```
Alba aparece en posición 0 en la lista  
Jimena aparece en posición 1 en la lista  
Marta aparece en posición 2 en la lista  
Paula aparece en posición 3 en la lista
```


Control de flujo: while

- * También existe el comando *while* (aunque no se use mucho).
- * La sintaxis es:

while condición:

orden_1

orden_2

```
x=1
while x<4:
    x=x+1
    print(x)
```

✓ 0.0s

2
3
4

Control de flujo: while

- * También existe el comando *while* (aunque no se use mucho).
- * La sintaxis es:

while condición:

orden_1

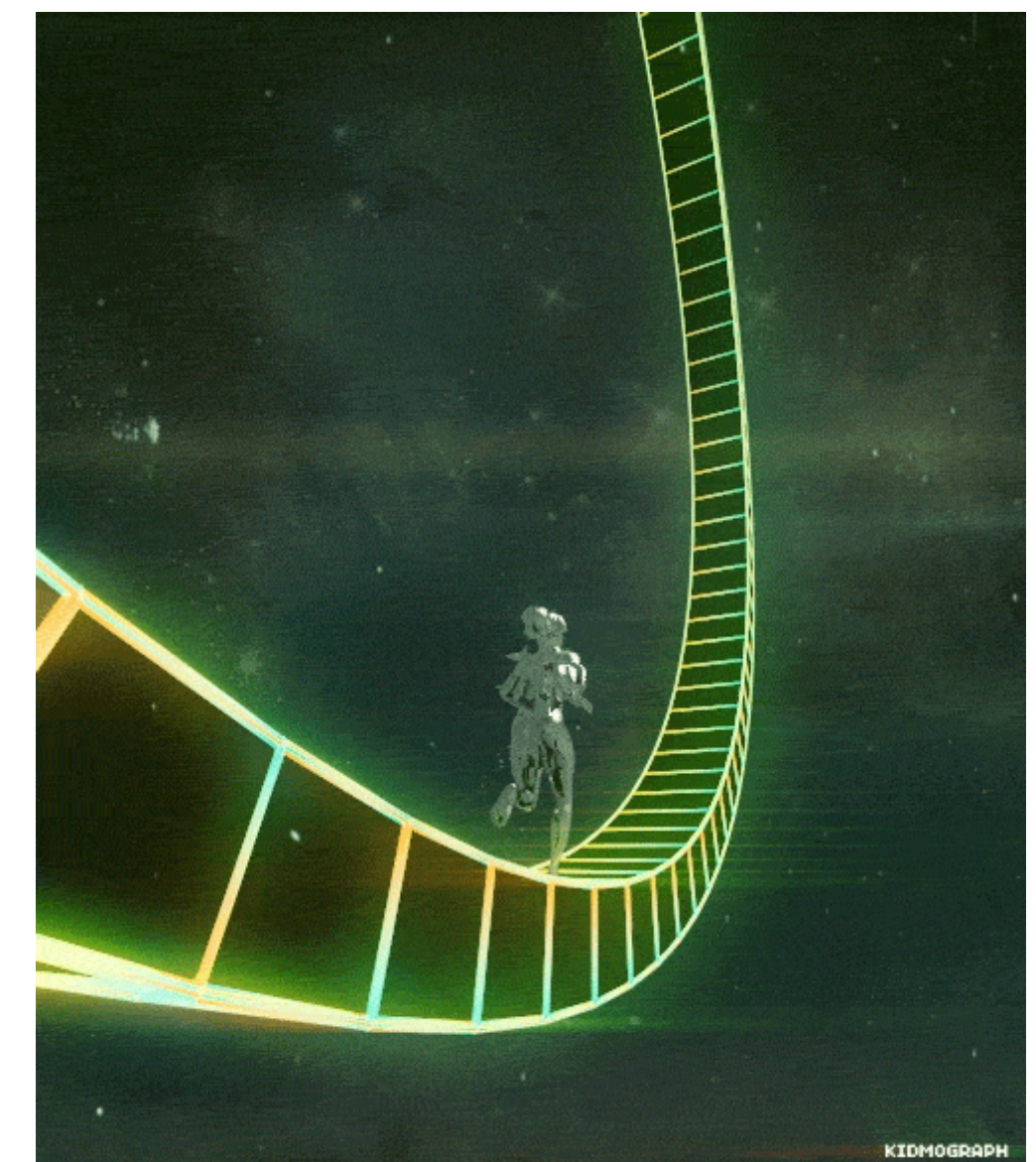
orden_2

```
x=1
while x<4:
    x=x+1
    print(x)
```

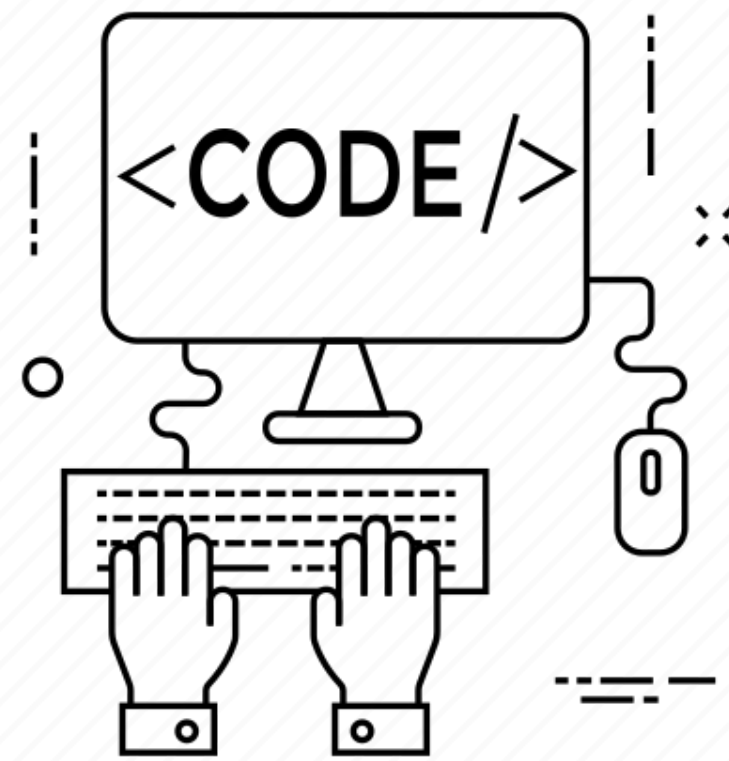
✓ 0.0s

2
3
4

CUIDADO
con los bucles infinitos



Manos a la obra



- * *Realizar un código que imprima en pantalla todos los números pares del 1 al 10 (incluido).*
- * *Explicar en una celda Markdown que el programa se hace para practicar el uso de for.*

Arrays

Listas y Arrays

- * *En Python se utilizan principalmente dos tipos de estructuras para manejar colecciones de datos: **listas** (listas estándar de Python) y **arrays** (de la biblioteca **NumPy**). Ambas tienen similitudes, pero se utilizan de manera ligeramente diferente.*
- * *¡En Python los ciclos for son muy lentos! Hay que intentar evitar el uso de ciclos.*
- * *Ejemplos de uso de listas:*

```
lista = [1, 2, 3, 4, 5] # Sintaxis para crear una lista
print(lista[0]) # Accede al primer elemento (índice 0)

lista[2] = 10 # Cambia el valor en la posición 2 (el 3 se convierte en 10)
print(lista) # [1, 2, 10, 4, 5]

lista.append(6) # Añade el número 6 al final
print(lista) # [1, 2, 10, 4, 5, 6]

lista.remove(10) # Elimina el valor 10 de la lista
print(lista) # [1, 2, 4, 5, 6]

lista2 = [7, 8, 9]
lista_completa = lista + lista2 # [1, 2, 4, 5, 6, 7, 8, 9]
print(lista_completa)

lista_repetida = lista2 * 2
print(lista_repetida)
```

✓ 0.0s

```
1
[1, 2, 10, 4, 5]
[1, 2, 10, 4, 5, 6]
[1, 2, 4, 5, 6]
[1, 2, 4, 5, 6, 7, 8, 9]
[7, 8, 9, 7, 8, 9]
```

Listas y Arrays

- * *NumPy permite trabajar con matrices y arrays de forma intuitiva y eficiente, especialmente cuando se trata de hacer operaciones matemáticas con vectores o matrices.*
- * *Un **array de NumPy** es una estructura similar a una lista, pero optimizada para trabajar con datos.*
- * *Ejemplos de uso: crear arrays y matrices*

```
import numpy as np # Import NumPy

arr1 = np.array([1, 2, 3, 4, 5]) # Creating a NumPy array from a Python list
arr2 = np.array([[1, 2, 3], [4, 5, 6]]) # Creating a 2D array (matrix)
zeros_arr = np.zeros(3) # Creating an array of zeros of size 3
ones_arr = np.ones((2, 4)) # Creating a 2x4 matrix of ones
print("Ones Array:\n", ones_arr)

range_arr = np.arange(1, 10, 2) # Creating an array with numbers from 1 to 9 with a step of 2
print("\nRange Array:", range_arr)
```

✓ 0.0s

Ones Array:
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]]

Range Array: [1 3 5 7 9]

Listas y Arrays

* Ejemplos de uso: acceder a elementos o subcadenas (slicing)

array[inicio : fin : paso]

```
arr1 = np.array([0, 1, 2, 3, 4]) # Creating a NumPy array from a Python list
arr2 = np.array([[1, 2, 3], [4, 5, 6]]) # Creating a 2D array (matrix)
print("First element of arr1:", arr1[0])
print("Element at row 1, column 2 of arr2:", arr2[1, 2])

print("Last element of arr1:", arr1[-1])
print("Slicing of arr1, elements from 2 to 4 (excluded):", arr1[2:4])
print("First 3 elements of arr1:", arr1[:3])
print("Last 3 elements of arr1:", arr1[-3:])
```

✓ 0.0s

```
First element of arr1: 0
Element at row 1, column 2 of arr2: 6
Last element of arr1: 4
Slicing of arr1, elements from 2 to 4 (excluded): [2 3]
First 3 elements of arr1: [0 1 2]
Last 3 elements of arr1: [2 3 4]
```


Listas y Arrays

- * *Tamaño de un array o matriz*

```
print("The length of arr1 is:", len(arr1))  
print("The shape of arr2 is:", arr2.shape)
```

✓ 0.0s

The length of arr1 is: 5

The shape of arr2 is: (2, 3)

Listas y Arrays

- * *Operaciones básicas con vectores y matrices*

```
# Basic operations
sum_arr = arr1 + 10 # Add 10 to each element
print("Array after adding 10:", sum_arr)

mult_arr = arr1 * 2 # Multiply each element by 2
print("Array after multiplying by 2:", mult_arr)

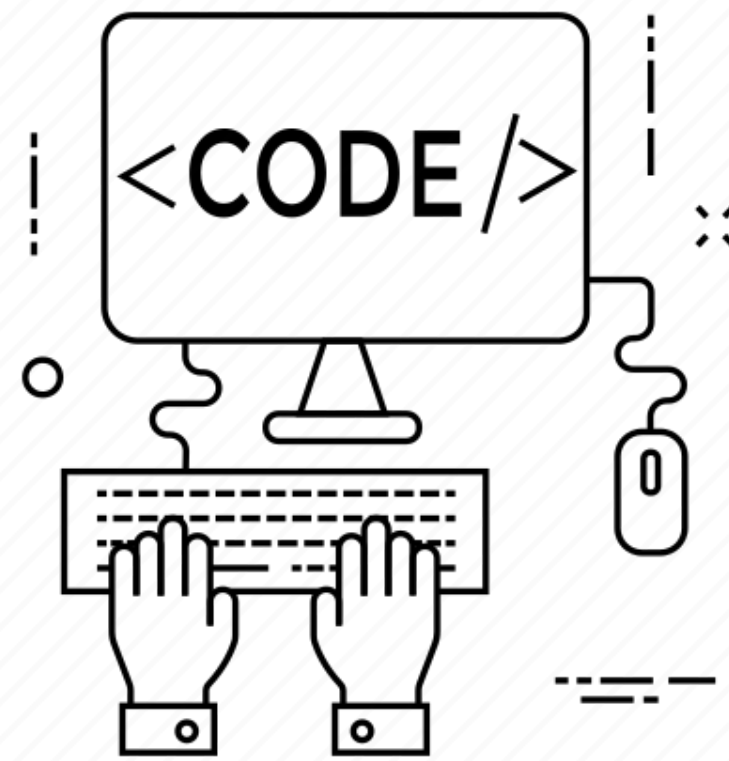
# Matrix multiplication
matrix_product = np.dot(arr2, arr2.T) # Multiply matrix with its transpose
print("\nMatrix product:\n", matrix_product)
```

✓ 0.0s

Array after adding 10: [10 11 12 13 14]
Array after multiplying by 2: [0 2 4 6 8]

Matrix product:
[[14 32]
[32 77]]

Manos a la obra



- * Crea un **array NumPy 5x5** con los números del **1 al 25** (ordenados en forma de matriz). Luego, realiza las siguientes operaciones usando **slicing**:
- * *Extrae la segunda fila completa;*
- * *Extrae la tercera columna completa;*
- * *Extrae la submatriz central (sin la primera y la última fila/columna);*
- * *Extrae los elementos de las esquinas (4 valores en total)*

```
import numpy as np

# Crear un array 5x5 con los números del 1 al 25
matriz = np.arange(1, 26).reshape(5, 5)
print(matriz)
```


Listas y Arrays

```
M = np.arange(1,26).reshape(5,5)
print(M)
print(M>14)
print(M[M>14])
```

✓ 0.0s

```
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]
 [11 12 13 14 15]
 [16 17 18 19 20]
 [21 22 23 24 25]]
[[False False False False False]
 [False False False False False]
 [False False False False  True]
 [ True  True  True  True  True]
 [ True  True  True  True  True]]
[15 16 17 18 19 20 21 22 23 24 25]
```


Input y output: ficheros

Input y output: ficheros

```
#write array to file
arr1=np.arange(5,1000,5)
np.savetxt("array.csv", arr1, delimiter=",")
#read array from file
arr_read = np.loadtxt("array.csv", delimiter=",")

print(arr_read[:10]) #print first 10 elements
```

✓ 0.0s

```
[ 5. 10. 15. 20. 25. 30. 35. 40. 45. 50.]
```

Input y output: ficheros

```
HV_cohete = np.loadtxt("altura_velocidad_cohete.txt", skiprows=1)
print(HV_cohete.shape) #print shape of array
print(HV_cohete[:4,:]) #print first 4 rows
```

✓ 0.0s

(101, 3)

```
[[ 0.    150.    100. ]
 [ 1.    245.096  90.193]
 [ 2.    330.386  80.386]
 [ 3.    405.868  70.579]]
```


Input y output: ficheros

```
HV_cohete = np.loadtxt("altura_velocidad_coheteX.txt",skiprows=1)
```

⊗ 0.0s

Python

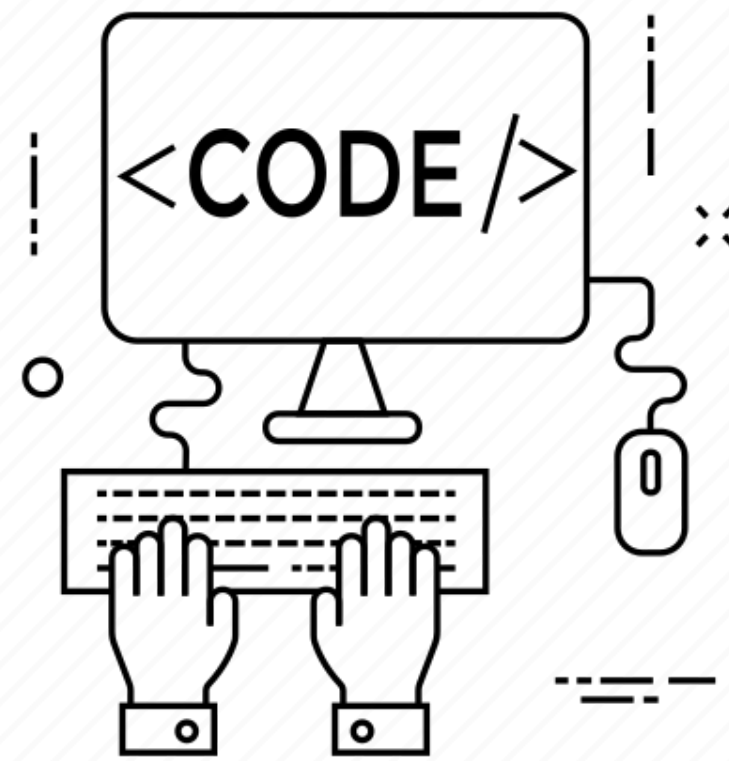
```
-----  
FileNotFoundError                                Traceback (most recent call last)  
/var/folders/g4/9bth9vr6nq9ddcv57rp6j680000gn/T/ipykernel_23162/144000287.py in <cell line: 1>()  
----> 1 HV_cohete = np.loadtxt("altura_velocidad_coheteX.txt",skiprows=1)
```


**NOT SURE IF C HAS TAUGHT ME
TO THINK LIKE A PROGRAMMER**

OR PYTHON IS JUST REALLY EASY

quickmeme.com

Manos a la obra



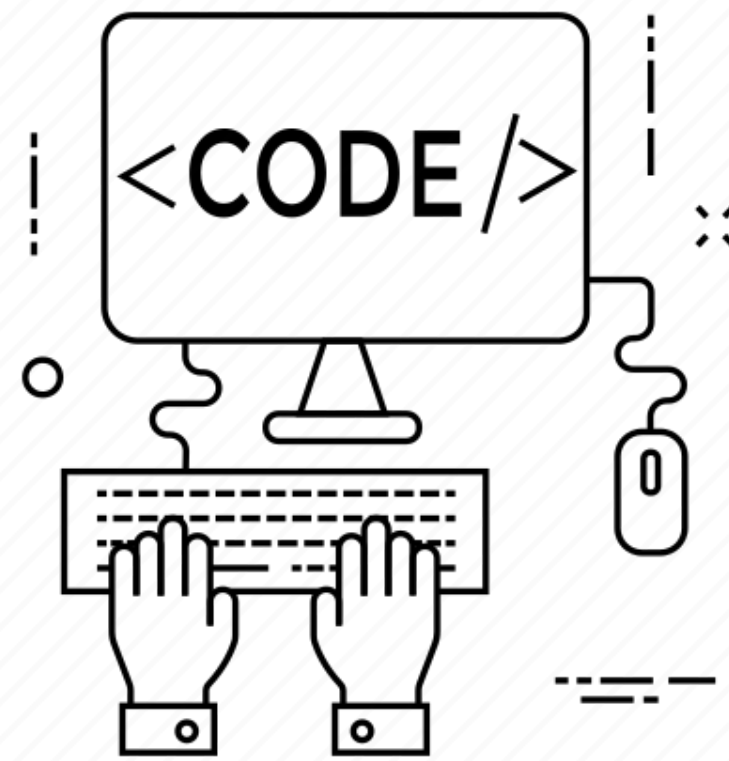
- * *Escribir una celda que genere un fichero que contenga los números enteros de -273 a 100 con saltos de 15*
- * *Escribir otra celda que lea ese fichero, transforme la temperatura a Fahrenheit y escriba por pantalla la temperatura. La relación entre Fahrenheit y Celsius es:*

$$F = \frac{9}{5}C + 32$$

temperaturas.txt

```
-273  
-258  
-243  
...
```

Manos a la obra



- * *Escribir un programa que*
 - 1) *lea el fichero Matriz_ejemplo.txt,*
 - 2) *lea el fichero Vector_ejemplo.txt*
 - 3) *calcule el producto matricial de M y v .*

Funciones

Funciones

- * *Una función es un bloque de código que solo se ejecuta cuando se llama.*
- * *Una función suele tener argumentos, conocidos como parámetros y puede devolver datos como resultado*

```
def una_funcion_ejemplo():  
    print("Hola desde dentro de la función!")  
✓ 0.0s
```

```
una_funcion_ejemplo()  
✓ 0.0s
```

Hola desde dentro de la función!

```
# Define a function to add two numbers.
```

```
def add_numbers(a, b):  
    """  
    This function takes two numbers as input and returns their sum.  
    """  
    return a + b # Returns the sum of a and b
```

```
sum = add_numbers(5,14)  
print(sum)
```

Funciones: ejemplos

```
# Define a function to calculate the square of a number
def square(num):
    return num ** 2

# Define a function to calculate the cube of a number
def cube(num):
    return num ** 3

# Define a function to calculate the factorial of a number
def factorial(num):
    if num == 0:
        return 1
    else:
        return num * factorial(num - 1)

# Test the functions
number = 5
print("The square of", number, "is", square(number))
print("The cube of", number, "is", cube(number))
print("The factorial of", number, "is", factorial(number))
```

✓ 0.0s

The square of 5 is 25
The cube of 5 is 125
The factorial of 5 is 120

Funciones: ejemplos

- * *Las funciones pueden devolver múltiples outputs*

```
def suma_y_producto(a, b):  
    # Esta función recibe dos números y devuelve su suma y su producto  
    suma = a + b  
    producto = a * b  
    return suma, producto  
  
# Llamamos a la función  
s, p = suma_y_producto(3, 4)  
  
print("Suma =", s)  
print("Producto =", p)  
✓ 0.0s
```

```
Suma = 7  
Producto = 12
```


Funciones: ejemplos

- * *Los argumentos de la función pueden tener valores **por defecto***
- * *Los argumentos obligatorios van primero (sin valores por defecto). La sintaxis es:*

```
def nombre_funcion(arg_obligatorio, arg_opcional=valor_por_defecto):  
    # Cuerpo de la función  
    return algo
```

```
def potential_energy(mass, height, gravity=9.81):  
    # Default gravity is set to Earth's gravity (9.81 m/s^2)  
    return mass * gravity * height
```

```
# Calling the function with 2 arguments  
print(potential_energy(10, 5))  
print(potential_energy(mass=10, height=5))
```

✓ 0.0s

490.50000000000006

490.50000000000006

```
# Calling the function with a custom gravity (e.g., Moon's gravity)  
print(potential_energy(10, 5, 1.62))
```

✓ 0.0s

81.00000000000001

Funciones: ejemplos

- * *Los argumentos de la función pueden tener valores **por defecto***
- * *Los argumentos obligatorios van primero (sin valores por defecto). La sintaxis es:*

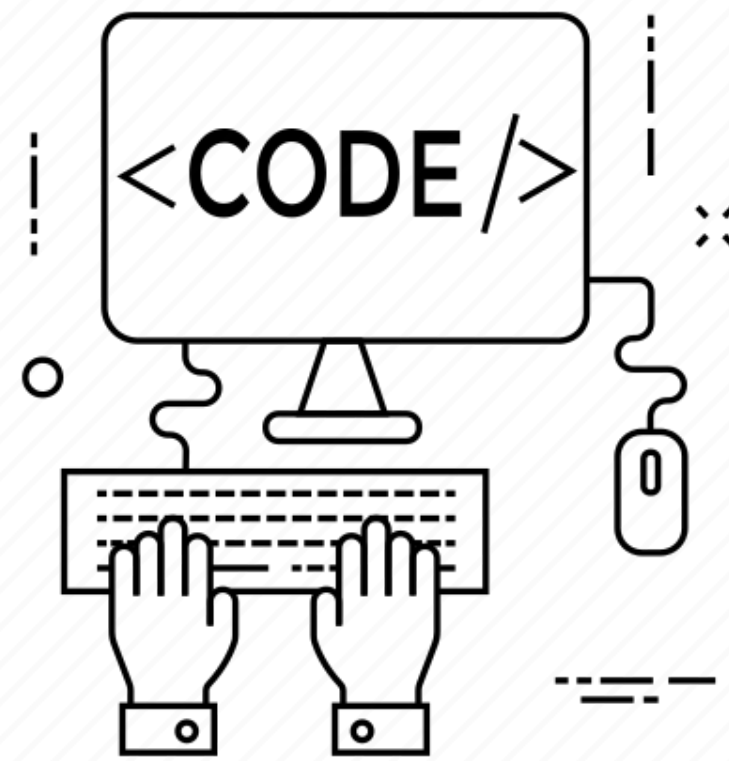
```
def nombre_funcion(arg_obligatorio, arg_opcional=valor_por_defecto):  
    # Cuerpo de la función  
    return algo
```

```
print(potential_energy(10,height=5))  
print(potential_energy(mass=10,gravity=1.62,height=5))  
✓ 0.0s  
490.50000000000006  
81.00000000000001
```

```
print(potential_energy(mass=10,5))  
⊗ 0.0s  
File "/var/folders/g4/9bth9vr6nq9ddcv57rp6j680000gn/T/ip  
    print(potential_energy(mass=10,5))  
                                ^  
SyntaxError: positional argument follows keyword argument
```

- * *Los argumentos cuyos nombres no se especifiquen tienen que estar al principio y en el orden en que aparecen en la definición de la función*

Manos a la obra



- * *Escribir un programa que genere un fichero con el encabezado “Temperatura(C)” que contenga los números enteros de -273 a 100 con saltos de 15 (un número por línea)*
- * *Escribir otro programa que lea ese fichero, transforme la temperatura a Fahrenheit y escriba por pantalla la temperatura. La relación entre Fahrenheit y Celsius es:*

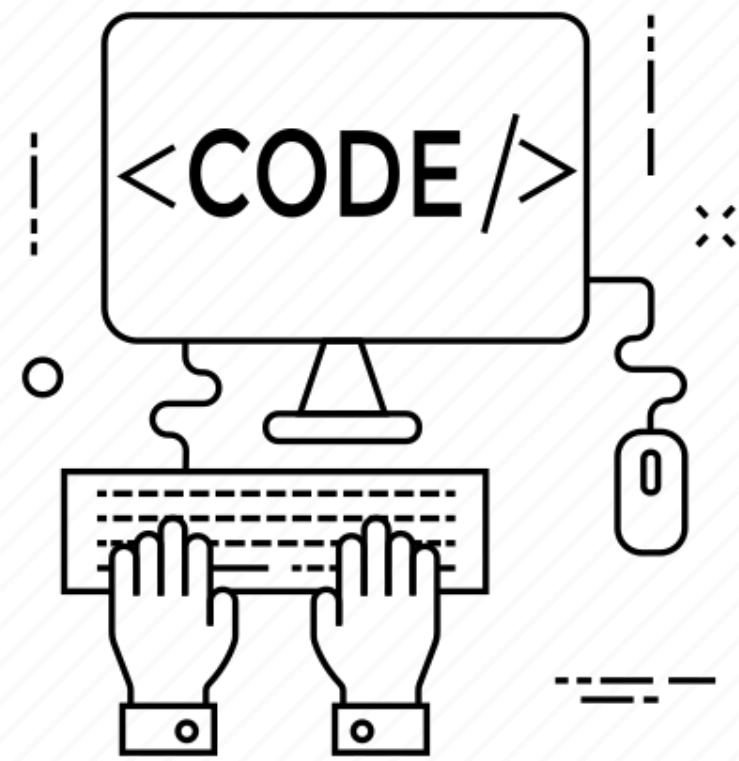
$$F = \frac{9}{5}C + 32$$

PRO TIP: chequea el uso de los argumentos de la función de numpy savetxt, en particular los argumentos **'header'**, **'comments'** and **'fmt'**.

temperaturas.txt

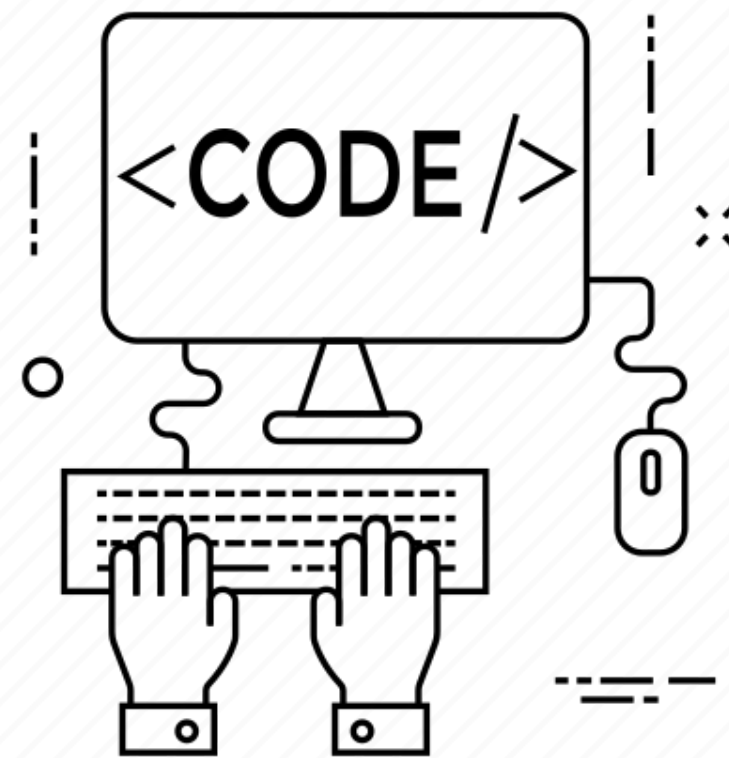
```
Temperatura(C)
-273
-258
-243
...
```

Manos a la obra



- * *Crear un programa que calcule la **velocidad final** de un objeto que cae libremente desde una cierta altura, en función de la altura.*

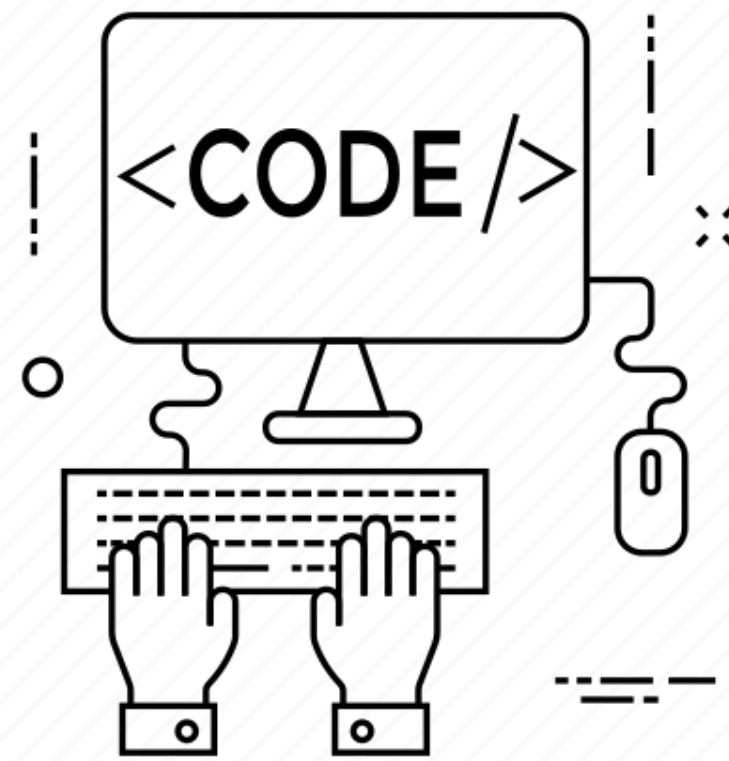
Manos a la obra



- * *Crear un programa que calcule la **velocidad final** de un objeto que cae libremente desde una cierta altura, en función de la altura.*

TIP para recordar la formula: cuál es la energía cinética del objeto en el instante inicial (cuando empieza a caer)? Y cuál es su energía potencial? Y en el instante final?

Manos a la obra



* *Escribir un programa que:*

- *defina un vector de tamaño $N=10$*
- *llame una función para llenar el vector con valores aleatorios entre 1 y 100*
- *llame una función para encontrar el valor mínimo del vector*
- *llame una función para imprimir por pantalla el contenido del vector*

PRO TIP: chequea el uso de la función de numpy **'random.randint'** y **'min'**.

Funciones de Arrays

Funciones de Arrays

- * *La sintaxis no cambia cuando el argumento o el output es un array o una matriz.*

```
# Esta función recibe un array y calcula dos funciones matemáticas
def funciones_trigonometricas(x):

    seno = np.sin(x)
    coseno = np.cos(x)

    return seno, coseno

# Creamos un array de valores entre 0 y 2π
x = np.linspace(0, 2*np.pi, 5)

print("\nValores de x:")
print(x)

# Llamamos a la función con el array
seno, coseno = funciones_trigonometricas(x)

print("\nsin(x):")
print(seno)

print("\ncos(x):")
print(coseno)
```

```
# Ejemplo de funciones que trabajan con matrices

# Definimos una función que recibe una matriz
# y devuelve su transpuesta, su traza y el producto M * M^T
def operaciones_matriz(M):

    transpuesta = M.T          # Transpuesta de la matriz
    traza = np.trace(M)         # Suma de los elementos de la diagonal
    producto = np.dot(M, M.T)  # Producto de la matriz por su transpuesta

    return transpuesta, traza, producto
```